

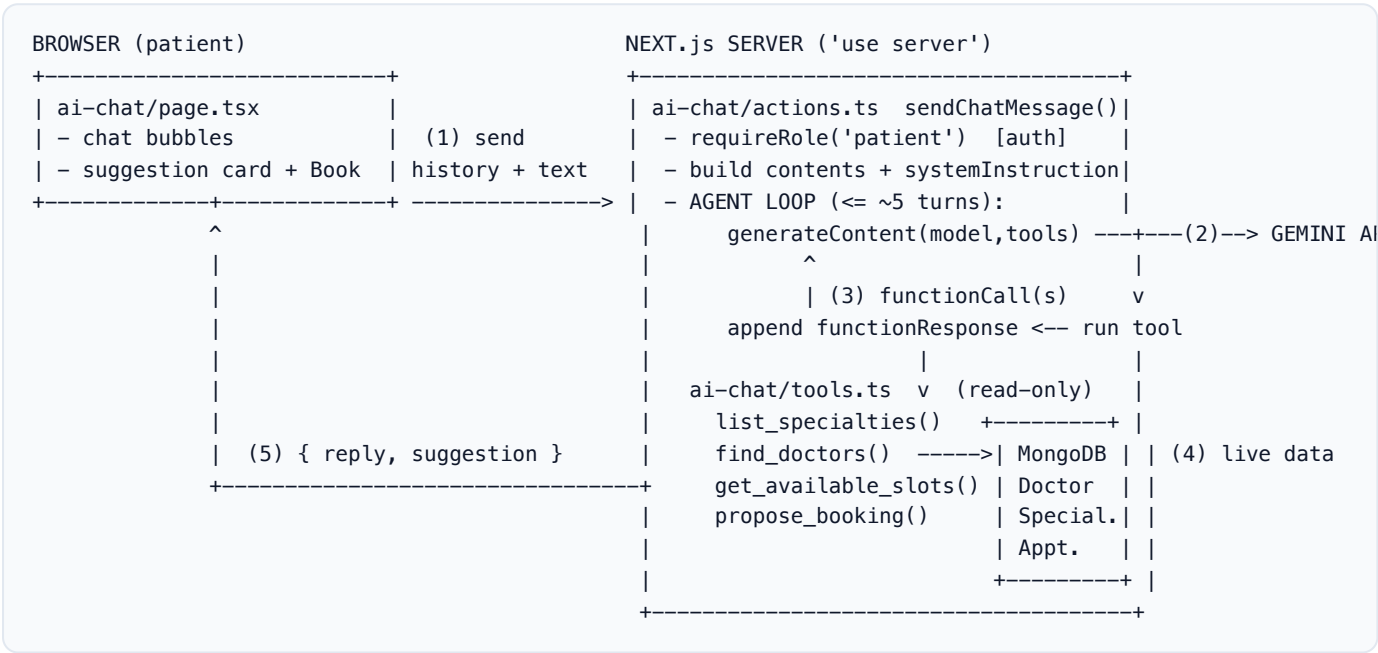
AI Integration — Solution Architecture

ClinicAI · Symptom → Doctor suggestion → Bookable slot

Provider: Google Gemini · Next.js 16 (App Router) · MongoDB / Mongoose · Prepared 2026-09-05

A patient describes a problem in plain language. The assistant determines the relevant specialty, suggests a real doctor from the clinic database, surfaces that doctor's live available slots, and links straight to booking — using the clinic's actual data, never invented doctors or times.

1 High-level flow



- 1. **1** — Client sends the full conversation + new message to the server action.
- 2. **2** — Server asks Gemini to respond, advertising the available tools.
- 3. **3** — Gemini replies with `functionCall(s)`, e.g. `findDoctors(specialty:"Cardiology")`.
- 4. **4** — Server executes the functions against MongoDB and feeds results back; loop repeats until Gemini produces a final answer.
- 5. **5** — Server returns `{ reply, suggestion }`; the UI renders the message and a Book link.

2 Layers & responsibilities

LAYER	FILE	RESPONSIBILITY	RUNS ON
UI	<code>ai-chat/page.tsx</code> (rewrite)	Chat bubbles, thinking state, suggestion card + Book link. Holds conversation in React state.	Client
Orchestration	<code>ai-chat/actions.ts</code> (new)	Auth guard, system prompt, function-calling agent loop, response shaping.	Server
Tools / data	<code>ai-chat/tools.ts</code> (new)	Functions Gemini can call + their JSON-schema declarations. All read-only.	Server
AI client	<code>lib/gemini.ts</code> (new)	<code>GoogleGenAI</code> client + single <code>GEMINI_MODEL</code> constant.	Server

LAYER	FILE	RESPONSIBILITY	RUNS ON
Domain reuse	<code>lib/schedule.ts</code> , models	Slot logic + DB models — unchanged, reused.	Server

Trust boundary. The `GEMINI_API_KEY` and all DB access live only on the server. The browser never sees the key, never talks to Gemini directly, and never queries the DB — which is why this is a server action, not a client-side SDK call.

3 The tools — the AI's knowledge of your clinic

The AI does not memorize your doctors; it looks them up through these functions. This keeps answers always-current and hallucination-resistant.

TOOL	INPUT	RETURNS (LIVE DB)	PURPOSE
<code>list_specialties</code>	—	Specialty names actually offered	Route symptom → a real specialty
<code>find_doctors</code>	<code>{ specialty }</code>	Active doctors: id, name, specialty, rating, experience	Pick a real doctor
<code>get_available_slots</code>	<code>{ doctorId, date }</code>	Real free "HH:MM" slots (schedule – booked)	Offer a real bookable time
<code>propose_booking</code>	<code>{ doctorId, date, time }</code>	Echoes structured data back to us	Clean Book link (no prose parsing)

The first three reuse existing logic in `lib/schedule.ts` (`availableSlots` , `dayOfWeek`) and the `bookedTimes` / `parseDay` helpers currently duplicated in the doctor and admin appointment actions (to be extracted into one shared helper).

4 Agent loop — why it's a loop, not one call

```
contents = [systemInstruction, ...history, userMessage]

repeat up to 5 times:
  resp = gemini.generateContent(contents, tools)
  if resp has functionCalls:
    for each call: result = runTool(call.name, call.args)
    append call + result to contents      # feed the data back
    continue                             # let Gemini reason on it
  else:
    return { reply: resp.text, suggestion: <from propose_booking> }
```

A single symptom → suggestion typically takes 2–4 model calls (list specialties → find doctors → get slots → answer). The cap of ~5 bounds cost and latency.

5 Design decisions (confirmed)

- **Provider:** Google Gemini · SDK `@google/genai` (requires Node ≥ 22).
- **Model:** Flash-Lite, pinned in one `GEMINI_MODEL` constant → one-line bump to Flash if routing quality is weak.
- **Data access:** Function calling (live DB), not prompt-dumping — accurate, scalable, extensible.
- **Runtime:** Stateless server action; client resends history each turn; no DB persistence yet (Recent Chats sidebar stays cosmetic).

Risk to watch. Flash-Lite is the weakest tier at the multi-step tool use this feature depends on. Mitigation: tight tool descriptions + system instruction, and the single model constant so upgrading to Flash is a one-line change.

6 Out of scope (for now)

- No conversation persistence / real chat history (stateless).
- No token streaming (reply arrives whole; a "thinking" spinner covers the wait).
- The AI **suggests and links to** booking — the patient confirms on the existing booking page. The AI does not silently create appointments.

7 Prerequisites & verification

Prerequisites

- `node -v` ≥ 22 , then `npm install @google/genai`.
- Add `GEMINI_API_KEY` to `.env` (Google AI Studio); read server-side only. `.env` is git-ignored.
- DB has ≥ 1 specialty and ≥ 1 active doctor **with a working schedule**, or the AI correctly reports no availability.

End-to-end checks

- Symptom ("chest pain climbing stairs") → routes to Cardiology, names a real doctor, lists real slots, shows a Book link.
- Vague input ("I feel unwell") → a brief clarifying question, not a random doctor.
- No doctor/slot in the reply that isn't in the DB (proves tool use, not hallucination). Book → lands on that doctor's page.
- `npx tsc --noEmit`, `npx eslint` on new files, `npx next build`.